

ANAMNESIS – An Information-Gain Driven Medical Chatbot for Symptom-Based Disease Triage

A PROJECT BASED LEARNING (PBL) REPORT

Submitted by

SAI CHARAN A - 2104251040844

ASHFAQ MUHAMMED A - 2104251040096

Submitted in partial fulfilment of the requirements

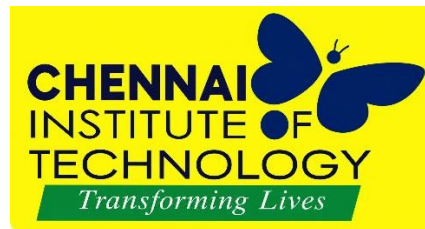
for the

Project-Based Learning component of Machine Learning

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI



Affiliated to Anna University, Chennai

(Autonomous)

OCTOBER 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

- IM1.** To create next generation leaders by effective teaching learning methodologies and instill Scientifics Park in them to meet the global challenges.
- IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.
- IM3.** To inculcate human values and ethical principles to cater the societal needs.
- IM4.** To contribute towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R & D establishments.
- IM5.** To nurture incubation centres enabling structured entrepreneurship and start-ups.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

- DM1.** To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.
- DM2.** To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.
- DM3.** To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

BONAFIDE CERTIFICATE

This is to certify that the Project-Based Learning report titled “**ANAMNESIS – An Information-Gain Driven Medical Chatbot for Symptom-Based Disease Triage**” is a Bonafide record of work carried out by **Sai Charan A [2104251040844]**, **Ashfaq Muhammed A [2104251040096]** of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor-guided Project-Based Learning (PBL) component of the Machine Learning course during the academic year [2026–2027] under my supervision.

SIGNATURE

Dr. S. PAVITHRA, M.E., Ph.D.,
Professor and Head,
Dept. of Computer Science and
Engineering
Chennai Institute of Technology,
Chennai – 69.

SIGNATURE

Ms. S. RAJA PRIYA, M.E.,
MENTOR
Assistant Professor
Dept. of Computer Science and
Engineering
Chennai Institute of Technology,
Chennai – 69.

Submitted for the final review held on

Internal Examiner

External Examiner

DECLARATION

I/We jointly declare that the PBL report on **“ANAMNESIS – An Information-Gain Driven Medical Chatbot for Symptom-Based Disease Triage”** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **“ANNA UNIVERSITY, CHENNAI”** for the requirement of Degree of **BACHELOR OF ENGINEERING**. This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING**.



SIRAGU

Signature

Sai Charan A

Ashfaq Muhammed A

Place: Chennai

Date: 02.10.2026

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honourable Chairman **SHRI. P. SRIRAM** for providing immense facilities at our institution.

We are very proud to render our thanks to our Principal **Dr. A. RAMESH M.E., Ph.D.**, for the facilities and the encouragement given by him toward the progress and completion of our project.

We would like to express special thanks and gratitude to our Dean **Dr. V. SRINIVASA RAO M.E., Ph.D.**, who has been a key source of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr. S. PAVITHRA M.E., Ph.D.**, for her effective leadership, encouragement and guidance throughout the project.

We would like to extend our thanks to the Project Co-ordinator **Ms. S. RAJA PRIYA M.E.**, Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from our class advisors **Ms. S. RAJA PRIYA M.E.**, and **Mr. R. RAHUL M.TECH.**, of the Department of Computer Science and Engineering for their valuable suggestions and support toward the successful completion of the project.

Sai Charan A (2104251040844)

Ashfaq Muhammed A (2104251040096)

ABSTRACT

Online symptom checkers are among the most widely used health tools, yet audits have found them to name the correct condition first in only about a third of cases, and most student and hobby projects in this space quote near-perfect accuracy that does not survive contact with a real patient. This project began by auditing the public disease–symptom dataset that most such projects use (4,920 records, 41 diseases, 132 binary symptoms) and found that it contains only 304 distinct records and that every one of the 41 rows in its official test file also appears in the training file, so the commonly reported 100% accuracy measures memorisation. Even under a leakage-free, profile-grouped split, standard classifiers stay near 100% when given the complete 132-symptom vector, but fall to 61.8% (logistic regression) when a patient volunteers only two symptoms, which is how real conversations begin. ANAMNESIS addresses that gap as a conversational problem rather than a classification one. A Bayesian evidence model conditions only on symptoms whose state is known, and at every turn the chatbot asks the single yes/no question with the highest expected information gain over the 41-disease posterior, stopping once one disease reaches 90% probability. Free-text replies are parsed by a lay-language symptom extractor with negation scope, warning-sign symptoms trigger an immediate emergency alert, and every answer is assembled from the knowledge base and the model’s own numbers, with no generative language model, so nothing the chatbot says can be invented. On 1,230 simulated consultations with held-out patient profiles and 3% answer noise, the system reached 96.3% top-1 and 98.7% top-3 accuracy after an average of 4.0 questions, against 90.4% for asking the most common symptoms and 79.4% for random questions, with an expected calibration error of 0.035. The symptom extractor reached an F1 of 0.927 on 50 hand-written lay-language utterances.

Keywords: Medical chatbot, Symptom checker, Bayesian inference, Expected information gain, Active questioning, Data leakage, Negation detection.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION.....	1
1.1 Background.....	1
1.2 Driving Question.....	1
1.3 Objectives.....	2
1.4 Scope and Limitations.....	2
1.5 Novelty and Contributions.....	3
CHAPTER 2: CONCEPT EXPLORATION.....	4
2.1 Related Approaches.....	4
2.2 Summary Table.....	5
2.3 What This Told Us.....	5
CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION.....	6
3.1 Weekly PBL Progress Log.....	6
3.2 Requirements.....	6
3.3 Feasibility.....	7
CHAPTER 4: ITERATIVE DESIGN AND DEVELOPMENT.....	8
4.1 System Architecture.....	8
4.2 Iteration 1 — Baseline.....	9
4.3 Iteration 2 — Refinement.....	9
4.4 Final Approach.....	10
4.5 Training Procedure.....	11
CHAPTER 5: IMPLEMENTATION.....	12
5.1 Module Description.....	12
5.2 Key Code Snippets.....	13
5.3 User Interface / Demo.....	14
CHAPTER 6: RESULTS AND DISCUSSION.....	18
6.1 Evaluation Metrics.....	18
6.2 Results Across Iterations.....	18
6.3 Discussion.....	20
6.4 Limitations.....	21
CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES.....	23
7.1 Individual Reflections.....	23
7.2 Team Learning.....	23
7.3 Course Outcomes — Evidence Summary.....	24
CHAPTER 8: CONCLUSION AND FUTURE SCOPE.....	25
8.1 Conclusion.....	25
8.2 Future Scope.....	25
REFERENCES.....	26

APPENDIX..... 27

A.1 Full Source Code.....27

A.2 Complete Weekly PBL Log.....27

A.3 Self and Peer Assessment.....27

A.4 PBL Poster.....28

LIST OF FIGURES

Figure 4.1 System Architecture Diagram.....	8
Figure 4.2 Information-Gain Questioning Loop.....	10
Figure 5.1 Consultation in Progress with Live Differential.....	14
Figure 5.2 Completed Consultation with Explanation and Triage.....	15
Figure 5.3 Warning-Sign Alert and Emergency Triage.....	15
Figure 5.4 Knowledge Question and Lay-Language Input with Negation.....	16
Figure 5.5 Phone-Width Layout.....	17
Figure 6.1 Accuracy Under Three Evaluation Protocols.....	19
Figure 6.2 Accuracy When the Patient Volunteers Only k Symptoms.....	19
Figure 6.3 Accuracy Against Number of Follow-up Questions.....	20
Figure 6.4 Reliability Diagram of Final Confidence.....	21

LIST OF TABLES

Table 2.1	Summary of Related Approaches.....	5
Table 3.1	Weekly PBL Progress Log.....	6
Table 3.2	Hardware and Software Requirements.....	7
Table 4.1	Model and Dialogue Configuration.....	11
Table 6.1	Leakage Audit of the Public Dataset.....	18
Table 6.2	Model Evaluation Results Across Iterations.....	19
Table 6.3	Question-Selection Policies (Early Stopping at 90%).....	20
Table 6.4	Symptom Extraction on Lay-Language Utterances.....	20

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ML	Machine Learning
PBL	Project-Based Learning
EIG	Expected Information Gain
NB	Naive Bayes
LR	Logistic Regression
RF	Random Forest
SVM	Support Vector Machine
k-NN	k-Nearest Neighbours
NLU	Natural Language Understanding
LLR	Log-Likelihood Ratio
ECE	Expected Calibration Error
TF-IDF	Term Frequency–Inverse Document Frequency
LLM	Large Language Model
API	Application Programming Interface
REST	Representational State Transfer
UI	User Interface
CSV	Comma-Separated Values

TEAM ROLES AND RESPONSIBILITIES

Both members jointly owned problem framing, the weekly mentor-review cycle, the literature exploration in Chapter 2 and the final report. The division below reflects primary ownership of the build; both members reviewed and tested the full system before each milestone.

Team Member	Primary Role	Key Responsibilities
Sai Charan A	Project Lead — ML Engine & Evaluation	Dataset leakage audit and grouped split; Bayesian evidence model (model.py); expected-information-gain question selection; dialogue manager and triage rules (engine.py); simulated-patient evaluation harness and all experiments (evaluate.py); FastAPI backend; report figures.
Ashfaq Muhammed A	NLU, Interface & Testing	Lay-language symptom lexicon and negation handling (nlu.py); the 50-utterance NLU test set; chat interface and live differential panel (HTML/CSS/JS); browser testing, screenshots and phone-width checks.

CHAPTER 1

INTRODUCTION

1.1 Background

For most people the first stop for a health worry is not a clinic but a phone. Symptom checkers — apps and chatbots that take a description of how someone feels and suggest what it might be and how urgently to act — are now used by millions, and in India, where the ratio of doctors to population remains low outside cities, they fill a real gap between doing nothing and travelling to a hospital. The best-known audit of these tools, however, found that across 23 commercial symptom checkers the correct diagnosis was listed first in only 34% of cases, and triage advice was appropriate in only 57% [1].

A large share of academic and student work in this area is built on a single public dataset of 4,920 patient records covering 41 diseases and 132 binary symptoms [11]. Projects built on it routinely report 100% accuracy for decision trees, random forests and naive Bayes alike. Such a result should have raised suspicion. In our first iteration we audited the data and found the explanation: the 4,920 rows are copies of only 304 distinct records, and every row of the official test file is also present in the training file. The models were being tested on answers they had already been shown.

Our larger observation came after removing that leakage. Given all 132 symptoms of an unseen patient, the 41 diseases remain almost perfectly separable, because each disease has a distinctive signature. But no patient opens a conversation by listing 132 symptoms. They say two or three things — “fever and chills, I keep vomiting” — and the clinician’s skill lies in deciding what to ask next. This history-taking is called **anamnesis**, and it gives the project its name. ANAMNESIS treats diagnosis as a sequence of well-chosen questions rather than a single classification.

1.2 Driving Question

The question the team set out to answer was: **“When a patient describes only a few symptoms in their own words, can a chatbot reach the right condition reliably by choosing its follow-up questions mathematically — and can it do so honestly, without inventing anything and without hiding the leakage that inflates published results?”**

This narrowed into a buildable task: audit and repair the evaluation protocol for the public dataset; measure how standard classifiers behave when most symptoms are unknown; build a probabilistic model that treats unknown symptoms as unknown rather than absent; select each next question by its expected information gain; and measure, on simulated patients the model has never seen, how accuracy grows with each question asked.

1.3 Objectives

- To audit the widely used disease–symptom dataset for duplication and train/test leakage, and to define a profile-grouped evaluation split in which no test record has a copy in training.
- To quantify how standard classifiers (logistic regression, random forest, SVM, naive Bayes, decision tree, k-NN) degrade when a patient reports only 1–4 symptoms, which is how real consultations begin.
- To design a Bayesian evidence model that conditions only on known symptoms, and an expected-information-gain policy that chooses each follow-up question to reduce diagnostic uncertainty as quickly as possible.
- To build a lay-language symptom extractor with negation handling, a warning-sign triage layer, and an explanation of every result in terms of the patient’s own answers.
- To deliver the system as a working web chatbot and evaluate it on top-1/top-3 accuracy, questions asked, calibration and robustness to answer noise.
- To document weekly progress and mentor feedback through the PBL cycle, and reflect on the team’s approach and division of work.

1.4 Scope and Limitations

In scope. A single-user web chatbot that accepts free-text symptom descriptions in English, asks yes/no follow-up questions, and returns the most consistent of 41 conditions with its probability, the evidence for and against it, a description, precautions and a triage level. It also answers “what is ...” questions about the 41 conditions from its knowledge base.

Out of scope. ANAMNESIS is not a diagnostic device and does not prescribe medicines or doses. It covers only the 41 conditions and 132 symptoms in its knowledge base and cannot recognise anything outside them. Evaluation uses simulated patients generated from held-out dataset profiles rather than real patients, and no clinician has reviewed its outputs; these limits are discussed in Section 6.4.

1.5 Novelty and Contributions

The project makes five contributions that distinguish it from the typical symptom-prediction project built on the same data:

- **A documented leakage audit.** We show that the dataset’s 4,920 rows reduce to 304 unique records and that 100% of the official test rows appear in training, and we replace the usual protocol with a profile-grouped split (Section 6.2).
- **Partial-evidence evaluation.** We measure accuracy when the patient volunteers only k symptoms — the condition that matters in a real conversation and that full-vector evaluation hides entirely.
- **Information-gain questioning.** The chatbot chooses every follow-up question by expected information gain over the disease posterior, reaching 96.3% top-1 accuracy in 4.0 questions on average, against 90.4% in 5.2 questions for a most-common-symptom strategy.
- **No generated text.** Every sentence is assembled from the knowledge base and the model’s own numbers. There is no large language model in the loop, so the chatbot cannot hallucinate a condition, a symptom or a treatment.
- **Visible reasoning.** A live panel shows the posterior over diseases, the remaining uncertainty in bits, the information value of each question asked, and a per-symptom likelihood-ratio explanation of the result.

CHAPTER 2

CONCEPT EXPLORATION

2.1 Related Approaches

2.1.1 Symptom checkers and static classifiers

Semigran et al. [1] audited 23 online symptom checkers with 45 standardised patient vignettes and found the correct diagnosis listed first in 34% of evaluations and within the top 20 in 58%, with appropriate triage advice in 57%. Their study set the realistic bar for this problem. Kononenko’s survey of machine learning for medical diagnosis [2] notes that naive Bayes, despite its independence assumption, is repeatedly competitive with far more complex learners on diagnostic tasks and — importantly for a medical tool — produces explanations clinicians can follow, because each symptom contributes a separable likelihood term.

2.1.2 Data leakage in machine-learning evaluation

Kaufman et al. [3] formalised leakage as information available at training time that would not be available when the model is used, and showed how easily it enters through careless splits. Kapoor and Narayanan [4] later found leakage affecting 294 published papers across 17 scientific fields, with duplicated records shared between training and test sets among the most common causes. Their taxonomy is exactly the lens that explained the 100% accuracies reported on the dataset used here.

2.1.3 Asking questions: sequential and conversational diagnosis

The idea of choosing the next observation to maximise expected information goes back to Lindley [5], who defined the information an experiment provides as the expected reduction in Shannon entropy [13]. In automatic diagnosis, Wei et al. [7] framed symptom inquiry as a task-oriented dialogue learned with deep reinforcement learning and showed that an agent that asks about additional symptoms outperforms a classifier given only the patient’s self-reported symptoms (accuracy 0.65 against 0.59). Peng et al. [6] (REFUEL) improved such agents with reward shaping and feature rebuilding to reach higher accuracy in fewer turns on simulated patients. These agents need many thousands of training episodes and offer little insight into why a question was asked.

2.1.4 Understanding the patient’s words

Patients rarely use clinical terms, and they say what they do not have as often as what they do. NegEx [8], a simple rule-based algorithm that detects negation cues and their scope in clinical text, reported 94.5% specificity and 84.5% positive predictive value — evidence that a carefully scoped rule-based approach to negation is both effective and auditable.

2.2 Summary Table

Ref.	Approach / Model	Dataset	Reported Result
[1]	23 commercial symptom checkers	45 standardised patient vignettes	Correct diagnosis first in 34%; appropriate triage in 57%
[4]	Survey of leakage in ML-based science	294 papers, 17 fields	Duplicates across train/test among the leading causes of inflated results
[7]	Deep-RL symptom inquiry dialogue	Medical dialogue corpus (MZ)	Accuracy 0.65 vs 0.59 for a classifier on self-reported symptoms
[6]	REFUEL: RL with reward shaping	Simulated patients (SymCAT)	Higher accuracy in fewer inquiry turns than the RL baseline
[8]	NegEx negation detection	Discharge summaries	Specificity 94.5%, PPV 84.5%
[2]	Naive Bayes and other learners for diagnosis	Several clinical datasets	NB competitive with complex learners and easy to explain

Table 2.1 — Summary of Related Approaches

2.3 What This Told Us

Three conclusions shaped the design. First, from [3] and [4]: before trusting any accuracy figure we had to check the data for duplicates, which is what exposed the leakage in Iteration 1. Second, from [7] and [6]: the gain in symptom checking comes from asking, not from a better one-shot classifier — but reinforcement learning was unnecessary for our purpose. With an explicit probabilistic model, Lindley’s expected information gain [5] can be computed exactly for every candidate question in under a millisecond, with no training episodes, and every question can be justified in bits. Third, from [2] and [8]: in a medical setting, explainability and auditable rules outweigh small accuracy gains from opaque models. We therefore chose a naive-Bayes-style evidence model, rule-based negation, and no generative language model anywhere in the answer path.

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

3.1 Weekly PBL Progress Log

The project ran across the twelve-week PBL cycle, from the Zeroth Review on 27 June 2026, where the team presented the problem statement, to the final evaluation and report. The table summarises milestones, work completed and the mentor feedback that shaped the next iteration.

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem framing (Zeroth Review, 27 June 2026)	Presented the problem statement: a medical chatbot that helps a user understand their symptoms and decide how urgently to seek care. Surveyed public symptom–disease datasets and selected the 41-disease, 132-symptom dataset.	Problem approved; asked the team to state clearly that the chatbot is not a diagnostic device and to define what it must refuse to do.
3–4	Concept exploration, baseline plan	Reviewed symptom-checker audits, leakage literature and conversational diagnosis (Chapter 2); planned a classifier baseline before any chatbot work.	Agreed with a measured baseline first; asked for a comparison of several classifiers rather than one.
5–6	Iteration 1 — classifier baseline	Trained six classifiers; all scored 100%. Audited the data: 4,920 rows reduce to 304 unique records and every official test row is in training. Built the profile-grouped split.	Asked the team to explain the 100% before claiming it; the leakage audit was accepted as a key finding.
7–8	Iteration 2 — partial evidence	Simulated patients who volunteer only 1–4 symptoms; showed the accuracy collapse; added masked-augmentation training, which recovered part of the loss.	Pointed out that real doctors ask questions rather than guess from two symptoms; suggested an interactive approach.
9–10	Final approach — information-gain chatbot	Built the Bayesian evidence model, expected-information-gain question selection, lay-language NLU with negation, warning-sign triage and the web chat interface.	Asked for emergency handling for dangerous symptoms and an explanation of why each result was given.
11–12	Evaluation, report, demo	Ran the full simulated-patient evaluation, noise and calibration studies and NLU test set; captured the interface; compiled this report.	Final system and evaluation accepted for the concluding review.

Table 3.1 — Weekly PBL Progress Log

3.2 Requirements

Dataset. The knowledge base is the public “Disease Prediction Using Machine Learning” dataset [11]: 4,920 training rows and 41 test rows, each a binary vector over 132 symptoms

labelled with one of 41 diseases, together with companion tables of disease descriptions, four precautions per disease and a 1–7 severity weight per symptom [12]. After de-duplication the training rows yield 304 unique symptom profiles, between 5 and 10 per disease. No other labelled data was used apart from the team-written NLU test set of 50 utterances (108 symptom labels).

Category	Requirement
Processor / RAM	Any dual-core CPU / 4 GB RAM; no GPU (developed on an Apple M-series laptop)
Programming language	Python 3.11+ (backend and experiments); HTML, CSS, JavaScript (interface)
Libraries / frameworks	NumPy, pandas, scikit-learn, Matplotlib (ML and evaluation); FastAPI, Uvicorn, Pydantic (server); Playwright (interface testing)
Knowledge source	Kaggle disease–symptom dataset with description, precaution and severity tables
Development environment	VS Code, terminal, Chromium browser
Version control	Git and GitHub (repository in Appendix A.1)

Table 3.2 — Hardware and Software Requirements

3.3 Feasibility

The project is light on computation and heavy on experimental design, which made it feasible in the PBL window on ordinary laptops. Fitting the evidence model is a single pass of counting over 304 records; scoring all 132 candidate questions by information gain takes about 0.18 ms per turn; and the complete evaluation of 1,230 simulated consultations under three policies, plus the noise sweep and every baseline, runs in under two minutes. The main time costs were the leakage investigation, writing the lay-language lexicon and its test set, and the interface, which were spread across the iterations in Table 3.1 so that frontend and model work could proceed in parallel.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

4.1 System Architecture

The system has an offline half that runs once at start-up and an online half that runs on every chat turn (Figure 4.1). Offline, the dataset is de-duplicated and the evidence model — a 41×132 table of the probability that a patient with each disease reports each symptom — is estimated, alongside the description, precaution and severity tables. Online, the patient's text is converted to symptom states, the posterior over diseases is updated, and a policy decides whether to raise a warning, conclude, or ask the most informative next question.

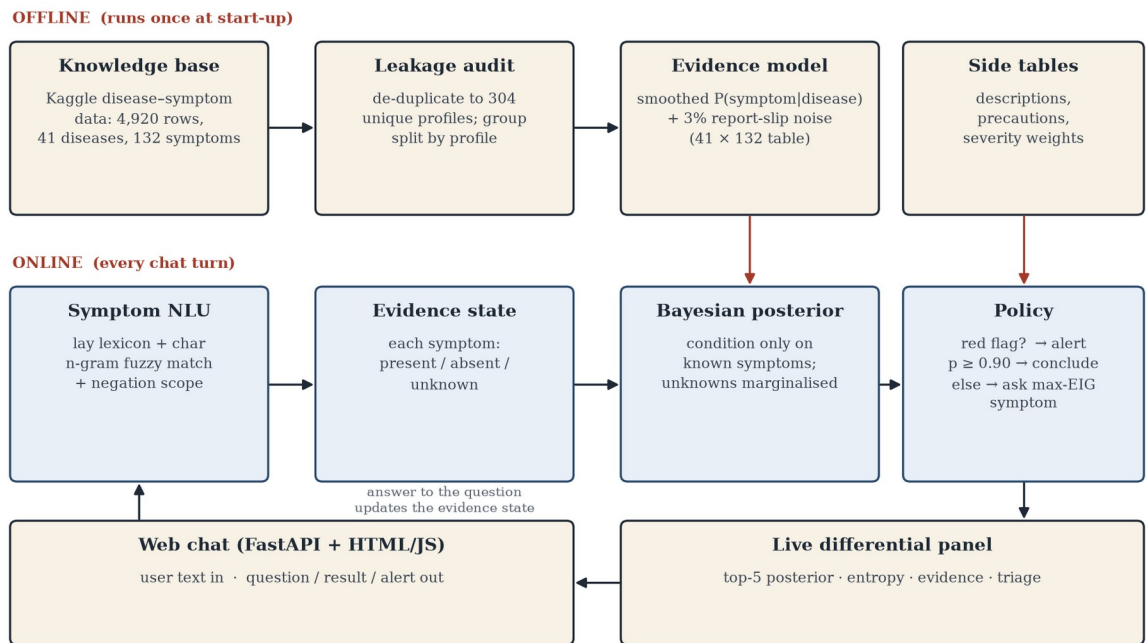


Figure 4.1 — System Architecture Diagram

Knowledge base and leakage audit: the 4,920 rows are reduced to 304 unique profiles before anything is learned, and every evaluation splits by profile so no test record has a copy in training.

Symptom NLU: free text is split into clauses; lay terms and misspellings are mapped to canonical symptoms by a lexicon and a character n-gram similarity match; negation cues mark symptoms as absent.

Evidence state and posterior: every symptom is present, absent or unknown. The posterior conditions only on present and absent symptoms; unknown symptoms are marginalised out.

Policy and delivery: warning-sign symptoms trigger an emergency alert; the conversation ends once one disease reaches 90% or questions run out; otherwise the highest-information question is asked. A FastAPI server delivers each turn to the browser, where a live panel shows the differential, evidence and triage.

4.2 Iteration 1 — Baseline

The simplest working version followed the usual recipe for this dataset: six scikit-learn classifiers (logistic regression, random forest, RBF-kernel SVM, Bernoulli naive Bayes, decision tree and 5-nearest neighbours) trained on the 132-symptom vectors. Evaluated on the official test file and on a random 80/20 split, **every classifier scored 100%**.

Following our mentor’s challenge to explain this, we audited the data. The 4,920 training rows contain only 304 distinct records — each repeated about 16 times — and 100% of the official test rows and 100% of the rows in a random split also occur in training. We therefore built a **profile-grouped split**: for each disease, two unique profiles are held out entirely, over five random seeds. Under this protocol the decision tree fell to 66.1% and naive Bayes to 86.8%, while logistic regression, random forest, SVM and k-NN stayed at 100% — showing that, given the full symptom vector, the diseases are genuinely separable.

The real limitation surfaced when we asked what the chatbot actually receives. A patient volunteers two or three symptoms, so the other 130 inputs are not “absent” but unknown. Feeding only two reported symptoms to the same classifiers dropped logistic regression to 61.8% and random forest to 61.8% (Table 6.2). This motivated Iteration 2.

4.3 Iteration 2 — Refinement

Iteration 2 attacked the partial-evidence problem in two ways. First, **masked augmentation**: each training profile was copied 20 times with a random 10–80% of its symptoms removed, teaching a logistic regression to recognise diseases from fragments. With two reported symptoms this raised accuracy from 61.8% to 79.7%. Second, an **evidence model** that never pretends to know unreported symptoms: a smoothed naive Bayes over present/absent/unknown states, which reached 69.8% from two symptoms.

Both are one-shot guesses, however, and neither came close to the 100% achievable with full information. The mentor review at this stage made the decisive point: a doctor faced with “fever and vomiting” does not guess — they ask. Masked LR is the better one-shot classifier,

but the evidence model has a property it lacks: for every possible question it can predict how the answer would change the diagnosis. That property is what the final approach is built on.

4.4 Final Approach

Evidence model. For disease d and symptom s , θ_{ds} is the smoothed fraction of the disease's profiles that include the symptom, $\theta_{ds} = (n_{ds} + \alpha)/(n_d + 2\alpha)$ with $\alpha = 0.05$, mixed with a 3% report-slip rate ϵ so that a single wrong answer cannot eliminate the true disease: $q_{ds} = \theta_{ds}(1 - \epsilon) + (1 - \theta_{ds})\epsilon$. Given evidence E , with P the set of present and A the set of absent symptoms,

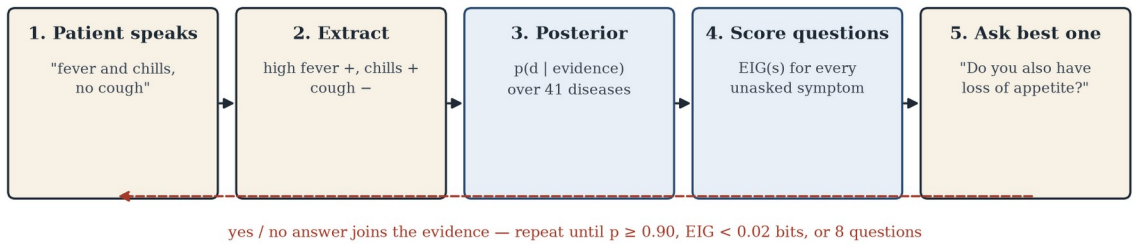
$$p(d | E) \propto p(d) \cdot \prod_{\{s \in P\}} q_{ds} \cdot \prod_{\{s \in A\}} (1 - q_{ds}),$$

and every unknown symptom simply drops out of the product. The prior $p(d)$ is uniform over the 41 diseases.

Expected information gain. For each unasked symptom s , the chatbot computes the probability of a “yes”, $P(\text{yes}) = \sum_d p(d|E) q_{ds}$, the two posteriors that would follow each answer, and the expected drop in entropy:

$$EIG(s) = H(D | E) - [P(\text{yes}) \cdot H(D | E, s = \text{yes}) + P(\text{no}) \cdot H(D | E, s = \text{no})]$$

and asks the symptom with the largest EIG. This is Lindley's criterion [5] applied exactly over all 132 symptoms in one vectorised NumPy operation. The conversation ends when the top disease reaches 0.90, when no question is worth at least 0.02 bits, or after eight questions. If the top disease is still below 0.50 at that point, the chatbot **abstains**: it names the three closest matches and recommends a doctor instead of naming one condition.



$$EIG(s) = H(D) - [P(\text{yes}) H(D | s = \text{yes}) + P(\text{no}) H(D | s = \text{no})], \quad P(\text{yes}) = \sum_d p(d) \theta_{ds}$$

Figure 4.2 — Information-Gain Questioning Loop

Explanation. For the final answer, each known symptom's contribution is reported as a likelihood ratio between the top disease and its nearest rival — for example, “pain behind the

eyes $\times 27.4$ ” means that answer was 27 times more likely under dengue than under the runner-up. **Triage.** Ten warning-sign symptoms (chest pain, breathlessness, slurred speech, one-sided weakness, altered sensorium, coma, blood in sputum, stomach bleeding, bloody stool, acute liver failure) trigger an immediate emergency alert regardless of the diagnosis; otherwise the triage level follows the leading condition (emergency, within 24 hours, or routine).

4.5 Training Procedure

The evidence model is fitted by counting, so there is no gradient training. Its two hyperparameters (α and ϵ) and the dialogue thresholds were set on training profiles only. For every reported result, the 304 unique profiles were split by profile into 222 training and 82 test profiles (two per disease) over five seeds; each test profile was used as a **simulated patient** three times, each time volunteering two randomly chosen symptoms from its profile and answering every follow-up question truthfully except for a 3% chance of a wrong answer. This gives 1,230 consultations per policy.

Component	Configuration
Data	304 unique profiles; grouped split 222 / 82 per seed; seeds 0–4; all random draws seeded
Evidence model	Smoothed Bernoulli likelihoods, $\alpha = 0.05$; report-slip $\epsilon = 0.03$; uniform prior over 41 diseases
Question policy	Maximum expected information gain over 132 symptoms (bits)
Stopping rule	Top posterior ≥ 0.90 , or best EIG < 0.02 bits, or 8 questions (10 in the evaluation)
Abstention	Name no single disease if the final top posterior < 0.50
Masked-augmented LR (Iter. 2)	20 masked copies per profile, keep-rate $U(0.2, 0.9)$; max_iter = 3000
Baselines (Iter. 1)	LR (max_iter 2000), RF (200 trees), SVM (RBF), Bernoulli NB, DT, k-NN ($k = 5$)
Simulated patient	2 volunteered symptoms; truthful answers with 3% flip noise (0%, 5%, 10% in sweep)
NLU	Lay lexicon (≈ 300 phrases) + char_wb 3–5-gram TF-IDF, cosine ≥ 0.72 ; clause-scoped negation

Table 4.1 — Model and Dialogue Configuration

CHAPTER 5

IMPLEMENTATION

5.1 Module Description

Backend — `anamnesis/`

- **knowledge.py** — loads the dataset and its description, precaution and severity tables; cleans symptom names; de-duplicates the 4,920 rows into unique profiles; maps misspelt disease names to display names.
- **model.py** — the EvidenceModel: fitting the smoothed likelihood table, the partial-evidence posterior, the vectorised expected-information-gain computation and the likelihood-ratio explanation.
- **nlu.py** — the SymptomExtractor: clause splitting, lay-term lexicon, character n-gram fuzzy matching, negation scope, and a yes/no reply parser.
- **engine.py** — the dialogue manager: one Session per conversation holding the evidence vector; red-flag alerts, stopping and abstention rules, triage levels and every message template.
- **app/server.py** — the FastAPI application exposing `/api/chat` and serving the interface.

Interface — `app/static/`

- **index.html**, **style.css** — the chat column and the live panel (Differential, Triage, Evidence), with a responsive single-column layout below 900 px.
- **app.js** — renders each message type (noted symptoms, question with Yes/No buttons, warning, result card, information card, abstention) and redraws the panel from each server response.

Experiments — `experiments/`

- **evaluate.py** — every number in Chapter 6: leakage audit, partial-evidence study, simulated-patient dialogues under three policies, noise sweep, calibration and NLU scoring; writes `results.json` and the figures. **nlu_testset.py** — the 50 hand-written utterances with gold labels.

5.2 Key Code Snippets

The partial-evidence posterior. Only present and absent symptoms enter the likelihood; unknown symptoms are ignored, which is what lets the chatbot reason from two complaints:

```
def log_likelihood(self, state):
    pres, absn = state == PRESENT, state == ABSENT
    lt, l1t = np.log(self.theta), np.log1p(-self.theta)
    return lt[:, pres].sum(1) + l1t[:, absn].sum(1)

def posterior(self, state):
    logp = np.log(self.prior) + self.log_likelihood(state)
    logp -= logp.max() # numerical stability
    p = np.exp(logp)
    return p / p.sum()
```

Code 5.1 — Partial-evidence Bayesian posterior (model.py)

Expected information gain for all 132 candidate questions at once:

```
def expected_information_gain(self, post, state):
    def H(p):
        p = np.clip(p, 1e-12, 1)
        return -(p * np.log2(p)).sum(0)
    q = self.theta # (diseases, symptoms)
    p_yes = post @ q # P(answer = yes) per symptom
    post_yes = (post[:, None] * q) / p_yes # posterior after "yes"
    post_no = (post[:, None] * (1 - q)) / (1 - p_yes)
    eig = H(post) - (p_yes * H(post_yes) + (1 - p_yes) * H(post_no))
    eig[state != UNKNOWN] = -np.inf # never re-ask
    return eig
```

Code 5.2 — Vectorised expected information gain (model.py)

Negation scope. A cue such as “no”, “without” or “don’t have” flips every symptom found after it in the same clause; clauses are split at punctuation and at “but”, so “headache but no fever” is read correctly:

```
NEGATION = re.compile(r"\b(no|not|never|without|none|denies|don't|do not|"
                      r"haven't|have not|free of|absence of)\b")
CLAUSE_SPLIT = re.compile(r"[.,;!?]\b|but\b|though\b|however\b")

neg = NEGATION.search(clause)
present = not (neg and neg.start() <= match.start())
```

Code 5.3 — Clause-scoped negation (nlu.py)

The turn policy in the dialogue manager:

```
post = model.posterior(self.state)
if post.max() >= CONFIDENT and n_present >= 2 or len(self.asks) >= MAX_QUESTIONS:
    return self._conclude(model, post) # result, or abstain if p < 0.50
eig = model.expected_information_gain(post, self.state)
j = int(np.argmax(eig))
if eig[j] < MIN_EIG_BITS:
    return self._conclude(model, post)
self.pending = j # ask: "Do you also have <symptom j>?"
```

Code 5.4 — Conclude-or-ask policy (engine.py)

5.3 User Interface / Demo

The interface is a single page with the conversation on the left and a live panel on the right. The patient types freely; the chatbot repeats back what it understood (“Noted: high fever; vomiting; chills; no cough”), and each follow-up question shows how many bits of information it is expected to yield, with one-tap Yes/No buttons. The panel updates after every answer: the top five diseases with their probabilities, the remaining uncertainty in bits, the evidence recorded so far (struck through when denied), each question asked with its information value, and the current triage level. All screenshots below were captured from the running application by an automated browser that typed and clicked exactly as a user would.

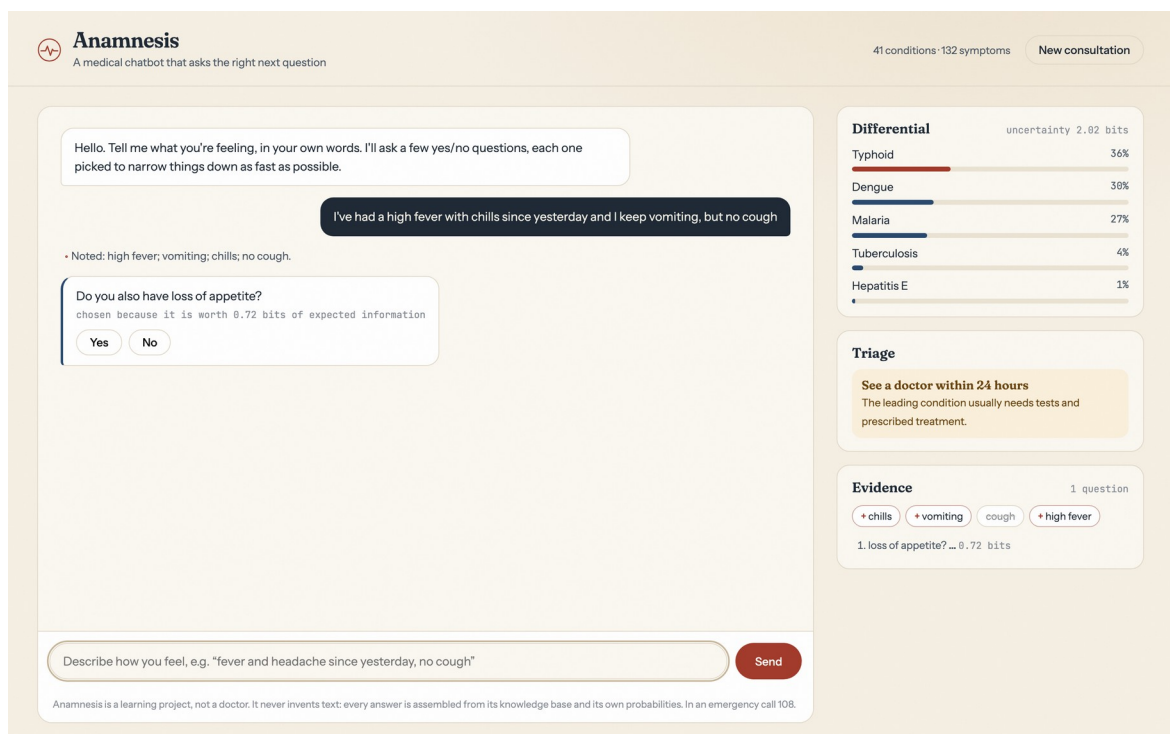


Figure 5.1 — Consultation in Progress with Live Differential

In Figure 5.1 the patient’s first message leaves typhoid, dengue and malaria nearly tied (36%, 30%, 27%) and 2.02 bits of uncertainty. The chatbot asks about loss of appetite because, of all 132 possible questions, its answer is expected to remove the most uncertainty (0.72 bits).

Anamnesis
A medical chatbot that asks the right next question

41 conditions · 132 symptoms [New consultation](#)

chosen because it is worth 0.72 bits of expected information

Do you also have pain behind the eyes?
chosen because it is worth 0.56 bits of expected information

yes

yes

Most consistent with
Dengue 99%
Next most likely: Tuberculosis (0.5%)

An acute infectious disease caused by a flavivirus (species Dengue virus of the genus Flavivirus), transmitted by aedes mosquitoes, and characterized by headache, severe joint pain, and a rash. — called also breakbone fever, dengue fever.

What points to it (vs. Tuberculosis)

- pain behind the eyes <27.4
- no cough >7.0

What argues against it

- Nothing reported counts against it.

Suggested precautions

- Drink papaya leaf juice
- Avoid fatty spicy food
- Keep mosquitos away
- Keep hydrated

Describe how you feel, e.g. "fever and headache since yesterday, no cough"

Send

Anamnesis is a learning project, not a doctor. It never invents text: every answer is assembled from its knowledge base and its own probabilities. In an emergency call 108.

Differential uncertainty 0.10 bits

Dengue	99%
Tuberculosis	0.5%
Typhoid	0.2%
Hepatitis E	0.1%
Malaria	0.1%

Triage

See a doctor within 24 hours
The leading condition usually needs tests and prescribed treatment.

Evidence 2 questions

- + chills
- + vomiting
- cough
- + high fever
- + loss of appetite
- + pain behind the eyes

1. loss of appetite? yes 0.72 bits
2. pain behind the eyes? yes 0.56 bits

Figure 5.2 — Completed Consultation with Explanation and Triage

After two answers (Figure 5.2) dengue reaches 99% and the chatbot concludes. The result card names the runner-up, explains which answers drove the decision as likelihood ratios, and shows the description and precautions from the knowledge base; the triage panel advises seeing a doctor within 24 hours.

Anamnesis
A medical chatbot that asks the right next question

41 conditions · 132 symptoms [New consultation](#)

Hello. Tell me what you're feeling, in your own words. I'll ask a few yes/no questions, each one picked to narrow things down as fast as possible.

Sudden chest pain, sweating a lot and I'm short of breath

• Noted: chest pain; breathlessness; sweating.

Warning sign
Chest pain can be a sign of something serious. If it is severe, sudden or getting worse, call 108 or go to the nearest emergency department now. I'll keep going, but don't wait on me.

Do you also have swelled lymph nodes?
chosen because it is worth 0.74 bits of expected information

Yes No

Describe how you feel, e.g. "fever and headache since yesterday, no cough"

Send

Anamnesis is a learning project, not a doctor. It never invents text: every answer is assembled from its knowledge base and its own probabilities. In an emergency call 108.

Differential uncertainty 1.61 bits

Pneumonia	38%
Tuberculosis	38%
Heart attack	25%
Common Cold	0.1%
Hypertension	0.1%

Triage

Seek emergency care now
Reported warning sign: breathlessness, chest pain

Evidence 1 question

- + breathlessness
- + sweating
- + chest pain

1. swelled lymph nodes? ... 0.74 bits

Figure 5.3 — Warning-Sign Alert and Emergency Triage

When the patient reports chest pain and breathlessness (Figure 5.3), a warning is raised immediately and the triage panel switches to “Seek emergency care now”, even though the differential still contains pneumonia and tuberculosis alongside heart attack — the safety rule does not wait for the model to be certain.

Anamnesis
A medical chatbot that asks the right next question

41 conditions · 132 symptoms [New consultation](#)

Hello. Tell me what you're feeling, in your own words. I'll ask a few yes/no questions, each one picked to narrow things down as fast as possible.

what is malaria?

Malaria
An infectious disease caused by protozoan parasites from the Plasmodium family that can be transmitted by the bite of the Anopheles mosquito or by a contaminated needle or transfusion. Falciparum malaria is the most deadly type.

Precautions

- Consult nearest hospital
- Avoid oily food
- Avoid non veg food
- Keep mosquitos out

my skin is itchy with a rash and some blistres, no fever

• Noted: itching; skin rash; no high fever.

Do you also have stomach pain?
chosen because it is worth 0.51 bits of expected information

☐ Yes ☐ No

Describe how you feel, e.g. "fever and headache since yesterday, no cough"

Send

Anamnesis is a learning project, not a doctor. It never invents text: every answer is assembled from its knowledge base and its own probabilities. In an emergency call 108.

Differential uncertainty 2.06 bits

Drug Reaction	43%
Fungal infection	39%
Chicken pox	7%
Psoriasis	2%
Chronic cholestasis	2%

Triage

Book a routine consultation
Nothing reported so far is a warning sign; monitor and see a doctor if it worsens.

Evidence 1 question

+itching +skin rash high fever

1. stomach pain? ... 0.51 bits

Figure 5.4 — Knowledge Question and Lay-Language Input with Negation

Figure 5.4 shows a “what is malaria?” question answered from the knowledge base, followed by everyday language — “my skin is itchy with a rash ... no fever” — parsed into itching, skin rash and, through negation, no high fever.



Anamnesis

A medical chatbot that asks the right next question

New consultation

ask a few yes/no questions, each one picked to narrow things down as fast as possible.

stomach ache and loose motions, not vomiting

• Noted: diarrhoea; stomach pain; no vomiting.

Do you also have irritability?
chosen because it is worth 0.64 bits of expected information

Yes No

Describe how you feel, e.g. "fe" Send

Anamnesis is a learning project, not a doctor. It never invents text: every answer is assembled from its knowledge base and its own probabilities. In an emergency call 108.

Differential

uncertainty 3.80 bits

Drug Reaction	26%
Hyperthyroidism	25%

Figure 5.5 — Phone-Width Layout

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Evaluation Metrics

Top-1 and top-3 accuracy measure whether the true disease is the chatbot’s first answer or within its three most probable. Top-1 suits the single condition shown on the result card; top-3 reflects the differential a clinician would read. **Mean questions asked** measures the burden on the patient — a symptom checker that needs twenty questions will be abandoned. **Expected calibration error (ECE)** [9] measures whether a stated 90% confidence is right about 90% of the time; this matters because the chatbot uses its own confidence to decide when to stop and when to abstain. For the symptom extractor, **precision, recall and F1** are computed over (symptom, present/absent) labels, so a negated symptom read as present counts as an error, and **negation accuracy** is reported separately.

All dialogue results use the profile-grouped split of Section 4.5, so every simulated patient is a symptom profile the model never saw, and every figure is the mean over five seeds. Accuracy alone would be misleading here: as Table 6.1 shows, the usual protocol reports 100% for models that have memorised the test set.

6.2 Results Across Iterations

Classifier (full 132-symptom input)	Official Testing.csv	Random 80/20 rows	Profile-grouped (ours)
Logistic Regression	100.0%	100.0%	100.0 ± 0.0%
Random Forest	100.0%	100.0%	100.0 ± 0.0%
SVM (RBF)	100.0%	100.0%	100.0 ± 0.0%
Bernoulli Naive Bayes	100.0%	100.0%	86.8 ± 1.2%
Decision Tree	100.0%	100.0%	66.1 ± 3.3%
k-NN (k=5)	100.0%	100.0%	100.0 ± 0.0%

Table 6.1 — Leakage Audit of the Public Dataset

Table 6.1 confirms the audit: the 41 official test rows and every row of a random split already appear in training, so the 100% column measures memory. Under a profile-grouped split the weaker learners fall, while LR, RF, SVM and k-NN remain perfect on complete vectors — the dataset is easy when every symptom is known, which is precisely why it says nothing about a conversation.

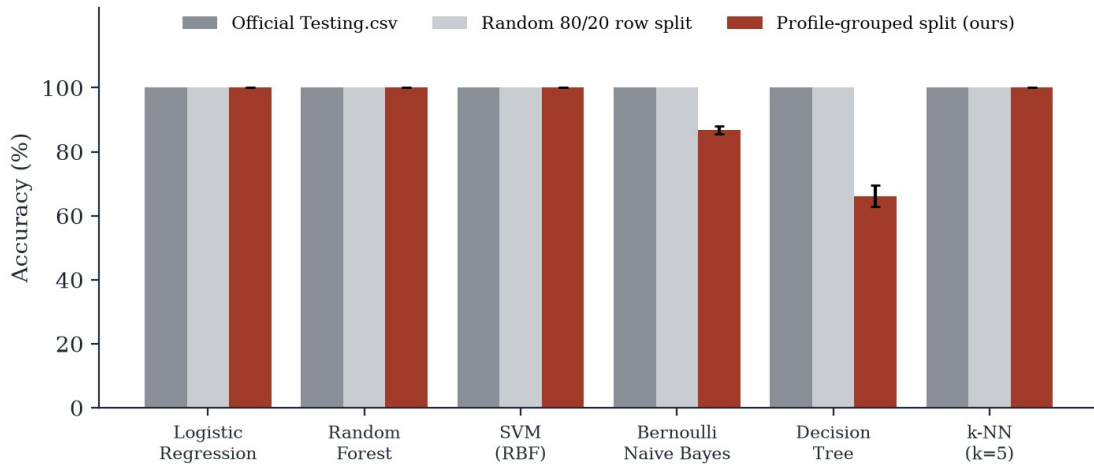


Figure 6.1 — Accuracy Under Three Evaluation Protocols

Version	Model	Input	Top-1	Top-3	Questions
Iteration 1 (baseline)	Logistic regression	2 volunteered symptoms	61.8%	—	0
Iteration 1 (baseline)	Random forest	2 volunteered symptoms	61.8%	—	0
Iteration 2 (refinement)	Masked-augmented LR	2 volunteered symptoms	79.7%	—	0
Iteration 2 (refinement)	Evidence model	2 volunteered symptoms	69.8%	90.2%	0
Final approach	Evidence model + EIG questioning	2 symptoms + follow-up answers	96.3%	98.7%	4.0

Table 6.2 — Model Evaluation Results Across Iterations

Table 6.2 is the project’s central result. Starting from the same two volunteered symptoms on unseen profiles, the best one-shot classifier reaches 79.7%; the final chatbot, by asking an average of 4.0 well-chosen questions, reaches 96.3% top-1 and 98.7% top-3. Figure 6.2 shows how every one-shot model depends on how much the patient happens to volunteer.

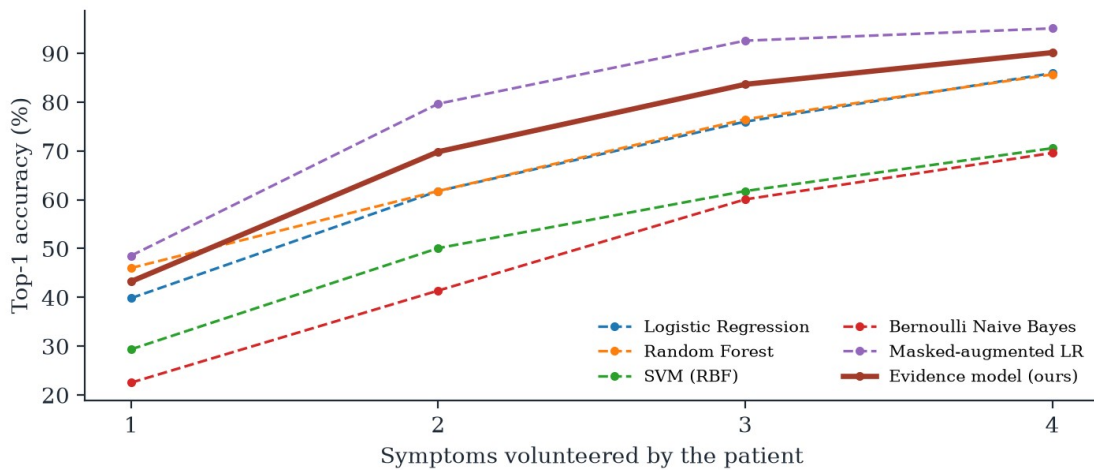


Figure 6.2 — Accuracy When the Patient Volunteers Only k Symptoms

Question-selection policy	Top-1	Top-3	Mean questions
Random unasked symptom	79.4%	94.8%	7.92
Most common symptom first	90.4%	98.5%	5.25
Expected information gain (ours)	96.3%	98.7%	4.02

Table 6.3 — Question-Selection Policies (Early Stopping at 90%)

Table 6.3 isolates the question policy, holding the model fixed. Information gain is both the most accurate and the least burdensome: 96.3% in 4.0 questions, against 90.4% in 5.2 for asking common symptoms first and 79.4% in 7.9 for random questions — 1.2 fewer questions than the next best policy, with higher accuracy.

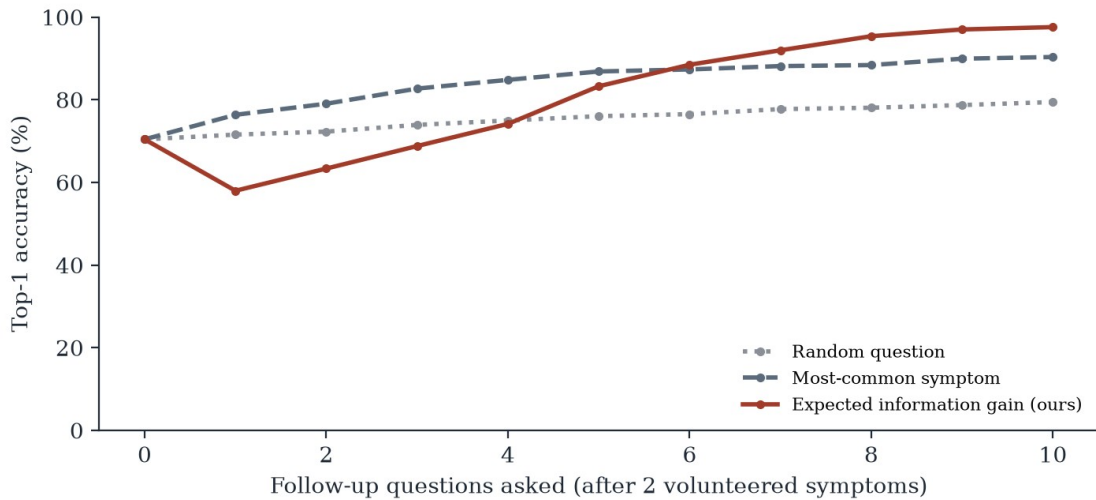


Figure 6.3 — Accuracy Against Number of Follow-up Questions

Extractor	Precision	Recall	F1	Negation accuracy
Exact canonical names only	0.974	0.352	0.517	58.3%
+ lay-term lexicon	0.957	0.824	0.886	91.7%
+ fuzzy char n-gram (final)	0.918	0.935	0.927	91.7%

Table 6.4 — Symptom Extraction on Lay-Language Utterances

On the 50 lay-language utterances (108 labels), matching canonical names alone finds barely a third of symptoms (recall 0.352), because patients say “throwing up”, not “vomiting”. The lay lexicon lifts F1 from 0.517 to 0.886, and fuzzy character n-grams, which catch misspellings such as “headach” and “diarhea”, raise it to 0.927 at a small cost in precision. Negation accuracy rises from 58% to 92%.

6.3 Discussion

Information gain is a long-term strategy. Figure 6.3 shows something we did not expect: after the first question, information gain is briefly less accurate (58.0%) than the starting point (70.4%), while asking common symptoms improves immediately. Information gain

deliberately asks the question that best splits the remaining candidates, which often spreads probability more evenly before concentrating it; the most-common-symptom policy confirms what is already likely. By the fifth question information gain overtakes it and keeps climbing to 97.6% at ten questions, while the frequency policy plateaus at 90.3%. With early stopping the chatbot never stops in that dip, because the posterior is not confident there.

The confidence can be trusted. The final confidence has an ECE of 0.035 (Figure 6.4), and 92.4% of consultations end above 0.90, where the observed accuracy is 97.8%. Abstaining below 0.50 withholds a single answer in only 3.2% of cases and lifts accuracy on the rest to 97.7%. Below 0.5 the model is under-confident rather than over-confident, which is the safe direction for a medical tool.

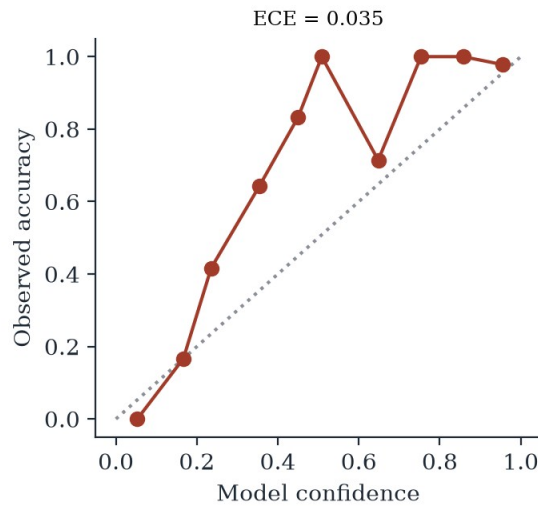


Figure 6.4 — Reliability Diagram of Final Confidence

Robustness to wrong answers. Patients misremember. With no answer noise the chatbot reaches 99.4% top-1; with 5% of answers flipped, 93.8%; with 10%, 87.2% — still 94.6% top-3 — while the mean number of questions rises only from 3.9 to 4.2. The 3% slip term in the model is what prevents one wrong answer from eliminating the true disease outright.

Why the evidence model and not masked LR. Masked augmentation is the better one-shot classifier (79.7% against 69.8% from two symptoms), but it cannot answer “what would I learn if I asked about joint pain?”. The evidence model can, for every symptom, in closed form, and that ability is worth far more than the one-shot gap.

6.4 Limitations

- **Simulated patients.** All dialogue results come from patients simulated from held-out dataset profiles. Real patients describe symptoms vaguely, have more than one

condition, and report symptoms outside the 132 used here; accuracy on real users will be lower.

- **A small, clean knowledge base.** 41 diseases and 304 unique profiles are far from clinical reality, and the dataset’s profiles are tidier than real presentations. The model inherits any errors in the source tables.
- **Uniform prior.** Every disease is treated as equally likely before the conversation; in practice the common cold is far more common than paralysis. Prevalence-aware priors would need epidemiological data we did not use.
- **No clinical validation.** No doctor has reviewed the questions, triage levels or warning-sign list, so the triage advice is a demonstration of mechanism, not a validated clinical rule.
- **English-only, rule-based NLU.** The extractor was tested on 50 team-written utterances; it does not handle Tamil or code-mixed input, and its negation rules can miss long-range or double negatives.

CHAPTER 7

TEAM REFLECTION AND LEARNING OUTCOMES

7.1 Individual Reflections

Sai Charan A: I led the project and built the machine-learning side — the leakage audit, the evidence model, information-gain question selection, the dialogue manager and the whole evaluation harness. The biggest lesson was that a 100% accuracy figure is a question, not an answer: the most important result of the project came from refusing to accept it and counting duplicate rows. The hardest part was designing an evaluation that is fair to a conversational system — simulated patients, grouped splits, answer noise — because no standard metric existed for “how good is a chatbot at asking questions”. I also learned that deriving a formula like expected information gain by hand and vectorising it in NumPy gave me a far deeper understanding than calling a library would have.

Ashfaq Muhammed A: I worked on the language and interface side — the lay-term lexicon, negation handling, the NLU test set and the chat interface with its live differential panel. Writing test utterances the way real people type showed me how little exact matching captures: canonical names alone found barely a third of the symptoms. The challenge was negation — “no fever” and “fever, no cough” look almost the same to a keyword matcher — and splitting at clause boundaries was the fix. On the interface, I learned that showing the probabilities and the information value of each question makes the system’s reasoning something a user can follow instead of a black box.

7.2 Team Learning

Splitting the work along the boundary between model and language worked because the two halves met at a simple interface: a list of (symptom, present/absent) pairs. Either side could change without breaking the other, and the NLU could be scored on its own test set, independently of diagnosis accuracy. Building the evaluation harness early in Iteration 2 also paid off — every later design choice, from the slip rate to the stopping threshold, was decided by a number rather than an impression.

Two pieces of mentor feedback changed our direction. The first, at the Iteration 1 review, was to explain the 100% accuracy before claiming it, which led to the leakage audit. The second, at the Iteration 2 review, was that a doctor asks rather than guesses — the remark that turned a classification project into a conversational one. If we restarted, we would audit

the dataset in week one, before training anything, and write the NLU test set before the lexicon, so that the lexicon could not be unconsciously tuned to the test.

7.3 Course Outcomes — Evidence Summary

- **Technical / ML competency:** six supervised classifiers, a hand-derived Bayesian evidence model, expected information gain implemented from its definition, masked data augmentation and calibration analysis (Chapters 4 and 6).
- **Problem framing and iteration:** the progression from a leaky 100% baseline to partial-evidence models to an active-questioning chatbot, each step driven by a measured failure and dated mentor feedback (Table 3.1).
- **Experimental rigour:** grouped splits, five seeds, simulated patients with answer noise, ablations of the question policy and the NLU components, and a deterministic, re-runnable evaluation script.
- **Teamwork and division of labour:** the roles table and Section 7.1 show a clear model/language split that both members jointly reviewed and tested before each milestone.
- **Responsible AI:** no generated text, warning-sign triage, abstention under uncertainty, explanations for every result, and a candid statement of limitations (Section 6.4).

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

ANAMNESIS shows that the hard part of a symptom checker is not classification but conversation. On the public dataset most such projects use, we showed that the widely reported 100% accuracy comes from duplicated records — 4,920 rows hold only 304 unique profiles and every official test row is in training — and that even honest classifiers fall to 61.8%–79.7% when a patient volunteers two symptoms. By modelling unreported symptoms as unknown and choosing each follow-up question by expected information gain, the chatbot reached 96.3% top-1 and 98.7% top-3 accuracy on unseen profiles in an average of 4.0 questions, with calibrated confidence (ECE 0.035), graceful degradation under noisy answers, and no generated text. The driving question is answered in the affirmative: well-chosen questions recover the accuracy that partial information takes away, and the system can be honest about how it got there.

8.2 Future Scope

- Pilot the chatbot with real users under clinical supervision, and have a doctor review its questions, triage levels and warning-sign list.
- Replace the uniform prior with region- and season-specific prevalence (for example dengue and malaria in the monsoon), which the Bayesian design supports without retraining.
- Extend the knowledge base to a larger disease–symptom resource and allow symptoms with severity and duration rather than yes/no only.
- Add Tamil and code-mixed Tamil–English input to the NLU, and speech input for users who find typing difficult.
- Make the question cost-aware, trading information against how uncomfortable or hard a question is to answer, and test learned (reinforcement-learning) policies against exact information gain.

REFERENCES

- [1] H. L. Semigran, J. A. Linder, C. Gidengil, and A. Mehrotra, “Evaluation of symptom checkers for self diagnosis and triage: audit study,” *BMJ*, vol. 351, p. h3480, 2015.
- [2] I. Kononenko, “Machine learning for medical diagnosis: history, state of the art and perspective,” *Artificial Intelligence in Medicine*, vol. 23, no. 1, pp. 89–109, 2001.
- [3] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [4] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, p. 100804, 2023.
- [5] D. V. Lindley, “On a measure of the information provided by an experiment,” *The Annals of Mathematical Statistics*, vol. 27, no. 4, pp. 986–1005, 1956.
- [6] Y.-S. Peng, K.-F. Tang, H.-T. Lin, and E. Chang, “REFUEL: Exploring sparse features in deep reinforcement learning for fast disease diagnosis,” in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018.
- [7] Z. Wei, Q. Liu, B. Peng, H. Tou, T. Chen, X. Huang, K.-F. Wong, and X. Dai, “Task-oriented dialogue system for automatic diagnosis,” in *Proc. 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 201–207.
- [8] W. W. Chapman, W. Bridewell, P. Hanbury, G. F. Cooper, and B. G. Buchanan, “A simple algorithm for identifying negated findings and diseases in discharge summaries,” *Journal of Biomedical Informatics*, vol. 34, no. 5, pp. 301–310, 2001.
- [9] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. on Machine Learning (ICML)*, 2017, pp. 1321–1330.
- [10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] Kaushil268, “Disease Prediction Using Machine Learning,” Kaggle dataset, 2020. [Online]. Available: <https://www.kaggle.com/datasets/kaushil268/disease-prediction-using-machine-learning>
- [12] itachi9604, “healthcare-chatbot: symptom description, precaution and severity tables,” GitHub repository. [Online]. Available: <https://github.com/itachi9604/healthcare-chatbot>
- [13] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [14] S. Ramírez, “FastAPI documentation.” [Online]. Available: <https://fastapi.tiangolo.com/> (accessed Sept. 2026).

APPENDIX

A.1 Full source code:

<https://github.com/MarcussRico/anamnesis-medical-chatbot>

The repository contains the chatbot (anamnesis/, app/), the evaluation harness (experiments/), the scripts that drew every figure and captured every screenshot (scripts/), the raw results (results/results.json) and instructions to reproduce each table with a single command.

A.2 Complete weekly PBL log and mentor sign-offs:

The full week-by-week log is given in Table 3.1 (Chapter 3); it spans the twelve-week PBL cycle beginning at the Zeroth Review on 27 June 2026.

A.3 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
Sai Charan A	60%	60%	Led the project; leakage audit, evidence model, information-gain dialogue, evaluation and backend
Ashfaq Muhammed A	40%	40%	Lay-language NLU and negation, NLU test set, chat interface and interface testing

